



AUDIT REPORT

October 2026

For



Table of Content

Executive Summary	03
Number of Security Issues per Severity	05
Summary of Issues	06
Checked Vulnerabilities	07
Techniques and Methods	09
Types of Severity	11
Types of Issues	12
Severity Matrix	13
■ Informational Severity Issues	14
1. Tokens sent to the token contract's own address are locked forever	14
Functional Tests	16
Threat Model	17
Automated Tests	27
Closing Summary & Disclaimer	28

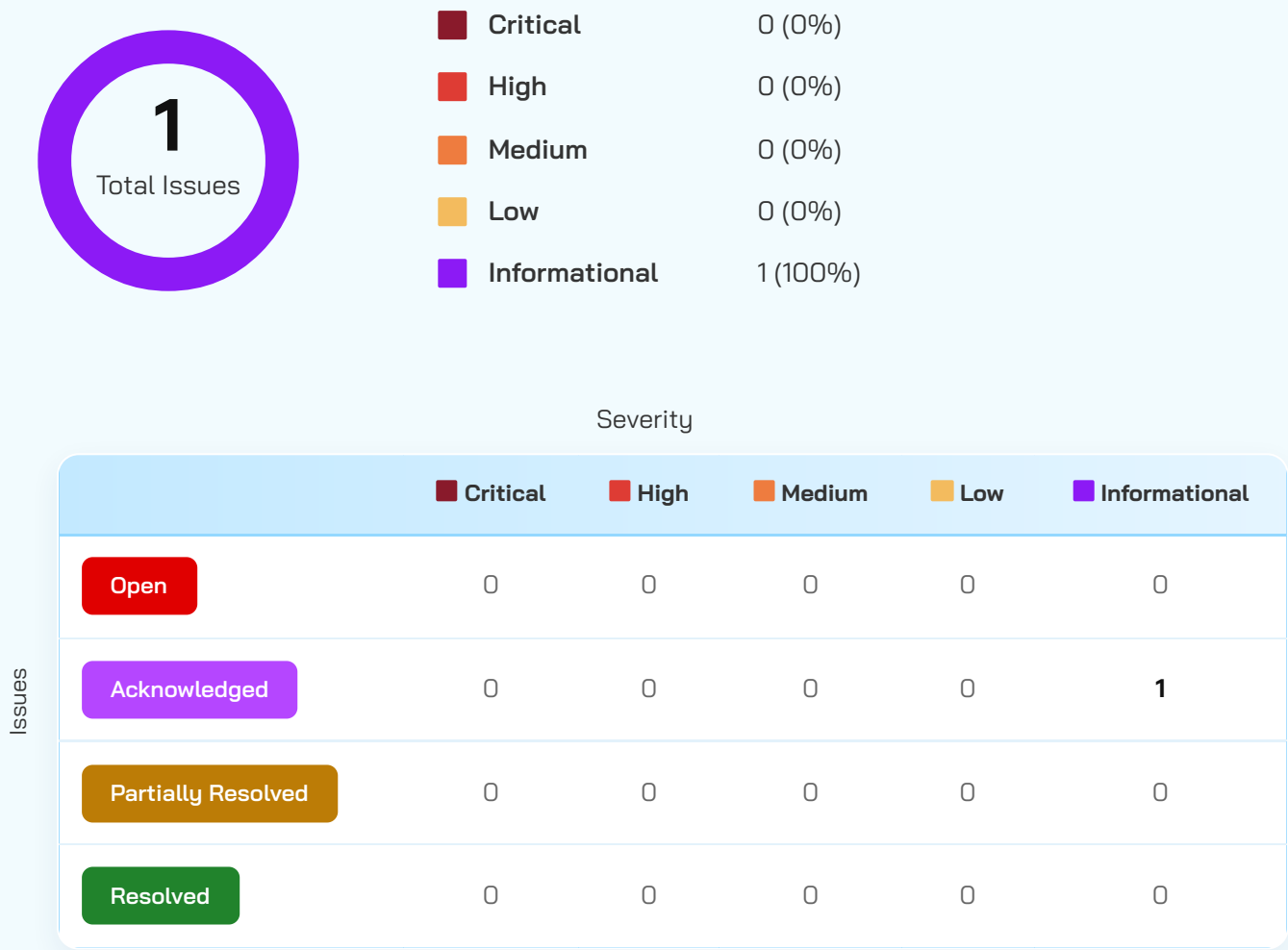
Executive Summary

Project Name	TBC Token
Protocol Type	ERC 20
Project URL	https://www.tenbestcoins.com/
Overview	TBC (TEN BEST COINS) is a standard ERC-20 token. The full supply of 25,000,000 is minted to a multi-sig treasury at deployment, which then distributes it to holders from the original token's snapshot using a batchTransfer helper.
Audit Scope	The scope of this Audit was to analyze the TBC Token Smart Contracts for quality, security, and correctness.
Source Code link	https://etherscan.io/address/0xfe4107D99F24d2318ff04E3DF44637503e146DEF#code
Contracts in Scope	TBCToken.sol
Branch	NA
Commit Hash	NA
Language	Solidity
Blockchain	EVM
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	7th October 2026
Updated Code Received	NA
Review 2	NA
Fixed In	NA

Executive Summary

✓ Verify the Authenticity of Report on QuillAudits Leaderboard: <https://www.quillaudits.com/leaderboard>

Number of Issues per Severity



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Tokens sent to the token contract's own address are locked forever	Informational	Acknowledged

Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls

✓ Missing Zero Address Validation

✓ Private modifier

✓ Revert/require functions

✓ Multiple Sends

✓ Using suicide

✓ Using delegatecall

✓ Upgradeable safety

✓ Style guide violation

✓ Using throw

✓ Unsafe type inference

✓ Using inline assembly

✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments, match logic and expected behavior.
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper.
- Implementation of ERC standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		■ High	■ Medium	■ Low
Likelihood	■ High	Critical	High	Medium
	■ Medium	High	Medium	Low
	■ Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Informational Severity Issues

Tokens sent to the token contract's own address are locked forever

Acknowledged

Path

```
contracts/TBCToken.sol
```

Function Name

```
transfer(), transferFrom() (inherited from ERC20) and batchTransfer()
```

Description

TBCToken accepts transfers to its own contract address. OpenZeppelin's `_transfer` only rejects the zero address as a recipient, and TBCToken adds no check of its own, so `transfer`, `transferFrom` and `batchTransfer` all send tokens to `address(this)` without reverting.

Once tokens are at that address, nothing can move them again. Moving tokens out of an address needs either a call from that address or an allowance from it. The token contract never calls `transfer` or `approve` on its own behalf, and because there is no owner, there is no rescue function and no way to add one later. The tokens stay counted in `totalSupply` but can never be spent.

This is a common mistake with ERC-20 tokens in general, because the token's contract address is the one holders copy most often, for example to add the token to a wallet or to look it up on a block explorer. It can also happen in bulk if the token's own address ends up in a `batchTransfer` recipient list.

Impact

None for the contract or for other holders. The only tokens at risk are the ones a holder sends to the wrong address by mistake, and those cannot be recovered.

Likelihood

Low. It needs a holder to make a mistake. No one else can cause it.

Tokens sent to the token contract's own address are locked forever*(continued)***Recommendation**

Reject transfers to the token's own address. In OpenZeppelin v5 this takes one override of the `_update` hook that every transfer passes through:

```
function _update(address from, address to, uint256 value) internal override
{
    require(to != address(this), "cannot send to token contract");
    super._update(from, to, value);
}
```

This is a fixed rule that nobody can change after deployment. It is not a blacklist and gives no address any special power. If the team prefers to keep the contract exactly as it is, the contract address should be published with a clear note that it is not a deposit address.

Functional Tests

TBCToken

- ✓ Should mint the full 25,000,000 TBC supply to treasury_ and nothing to the deployer
- ✓ Should revert deployment when the initial supply is zero, when treasury_ is the zero address, or when the supply would overflow
- ✓ Should not expose any owner, admin, mint, burn, pause, blacklist, guardian, fee or upgrade function to any caller, including the deployer and the treasury
- ✓ Should reject plain Ether sent to the token contract
- ✓ Should keep totalSupply fixed and equal to the sum of all balances across random sequences of transfers, approvals, permits and batch transfers (invariant test)
- ✓ Should not let the treasury move another holder's tokens without an allowance
- ✓ Should handle transfer, approve and transferFrom as the ERC-20 standard requires, including zero-amount transfers, self transfers and an unlimited allowance that is never reduced
- ✓ Should set an allowance through permit from a valid signature submitted by any address, and increase the holder's nonce
- ✓ Should reject a permit signature that is expired, reused, signed by the wrong key, malleable, or made for another deployment or another chain
- ✓ Should distribute whole and fractional amounts to multiple recipients in a single batchTransfer call from the caller's own balance
- ✓ Should revert the whole batch when the arrays have different lengths, when there are more than 300 recipients, when a recipient is the zero address, or when the caller's balance runs out partway through
- ✓ Should send to exactly 300 new recipients in one batchTransfer using about 7.7M gas, below the 16,777,216 gas per-transaction cap (EIP-7825)
- ✓ Should give the same balances with batchTransfer as with the same transfers sent one by one (fuzz test)

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

TBCToken is a standalone token. It does not call any external contract, holds no other project's tokens, and reads no oracle or price feed. Everything it depends on is either an OpenZeppelin base contract compiled into it or the compiler itself. There is no owner or admin, so once the contract is deployed no address has any power over it. The only trust boundary that matters is the treasury address that receives the whole supply.

1.1 External Dependencies Table

Dependency	Type	How Used	Trust Assumption	Risks
ERC20 (OpenZeppelin v5.6.1)	Base Contract	Provides balances, allowances, the transfer / approve / transferFrom entry points and the internal _update hook. TBCToken uses it unmodified and overrides none of its functions	_update is the single point every mint and transfer passes through	None identified. Every address is treated the same way

Dependency	Type	How Used	Trust Assumption	Risks
ERC20Permit, EIP712 and Nonces (OpenZeppelin v5.6.1)	Base Contract	EIP-2612 permit: an allowance set from an off-chain signature, with a nonce per holder and an EIP-712 domain tied to the chain id and the token address	Each signature is checked correctly and can be used only once	Only plain-key (ECDSA) signatures are accepted, so a Gnosis Safe treasury cannot use permit and has to call approve. A contract that calls permit and then transferFrom in one transaction can be front-run with the same signature, so integrators should not let a failed permit block the second step
ECDSA, MessageHashUtils, ShortStrings and Strings (OpenZeppelin v5.6.1)	Library	Signature recovery for permit (malleable high-s signatures are rejected) and storage of the EIP-712 name and version	Library code is correct	None identified
Context (OpenZeppelin v5.6.1)	Library	Supplies _msgSender() to the inherited ERC20 functions	Returns the real transaction caller	N/A
IERC20 / IERC20Metadata / IERC20Permit / IERC20Errors / IERC5267	Interface	Standard ERC20 and EIP-2612 function signatures, events and custom error types	Interfaces match the standards	N/A

Dependency	Type	How Used	Trust Assumption	Risks
Solidity compiler 0.8.24	Toolchain	Built-in overflow and underflow checks on all arithmetic	Compiler arithmetic checks are correct	None identified. The version is pinned, and its known compiler bugs do not apply to this code. OpenZeppelin v5.6.1 only compiles with evmVersion: cancun on this compiler, so the same setting must be used to verify the contract on the block explorer. The deployed bytecode itself uses no Cancun-only opcodes
Treasury address (multi-sig)	Off-chain actor	Receives the whole supply at deployment and distributes it with batchTransfer or a helper contract such as Disperse	The address is a Safe that exists on the deployment chain, is controlled by the team, and distributes the tokens as announced	The contract cannot check what treasury_ is. A wrong address, a single-key wallet, or a Safe that only exists on another chain puts the whole supply at risk, and with no owner the only fix is to redeploy before distribution starts. Tokens the treasury does not distribute stay with it

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

Contract Name	Function	Category
TBCToken.sol	constructor()	Main invariant(s) Total supply is fixed at deployment; there is no mint or burn function afterwards. initialSupply_ is in whole tokens and is multiplied by 10^{18} Access level Implicit restriction (runs once at deployment) What this function can do Set the name and symbol, set up the EIP-712 domain for permit, and mint the full supply to the treasury_ address passed in What this function cannot/should not do Run twice, mint to the deployer, or accept a zero supply or a zero address as treasury

Contract Name	Function	Category
TBCToken.sol	batchTransfer(address[], uint256[])	Main invariant(s) Both arrays are the same length; every leg goes through the public transfer, so the whole batch reverts if any one leg fails Access level Public, open to anyone (it only ever spends the caller's own balance) What this function can do Send tokens from the caller's own balance to up to 300 recipients in one call What this function cannot/should not do Move tokens the caller does not own, use an allowance, give the caller any special right, or exceed MAX_BATCH_SIZE
TBCToken.sol	transfer(address, uint256)	Main invariant(s) The sender's balance falls and the recipient's rises by the same amount; totalSupply never changes. The token's own address is accepted as a recipient (Informational issue 1) Access level Public (inherited from ERC20) What this function can do Move the caller's own tokens to another address What this function cannot/should not do Move more than the caller's balance, or send to the zero address

Contract Name	Function	Category
TBCToken.sol	approve(address, uint256)	Main invariant(s) Replaces the previous allowance; an allowance of type(uint256).max is never reduced when spent Access level Public (inherited from ERC20) What this function can do Let another address spend up to a set amount of the caller's tokens What this function cannot/should not do Move any tokens by itself, or approve the zero address
TBCToken.sol	transferFrom(address, address, uint256)	Main invariant(s) No address, including the treasury and the deployer, can move a holder's tokens without that holder's allowance Access level Public, requires an allowance (inherited from ERC20) What this function can do Move tokens from a holder that has approved the caller, and reduce the allowance by the same amount What this function cannot/should not do Spend more than the allowance or the holder's balance

Contract Name	Function	Category
TBCToken.sol	<pre> permit(address, address, uint256, uint256, uint8, bytes32, bytes32) </pre>	Main invariant(s) Each signature works once, because the holder's nonce goes up by one; the EIP-712 domain is tied to the chain id and the token address Access level Public (inherited from ERC20Permit) What this function can do Set an allowance from a holder's off-chain signature; anyone can submit it and pay the gas What this function cannot/should not do Accept a signature that is expired, already used, malleable, signed by someone other than the holder, or made for another chain or another deployment
TBCToken.sol	<pre> View functions: name, symbol, decimals, totalSupply, balanceOf, allowance, nonces, DOMAIN_SEPARATOR, eip712Domain, MAX_BATCH_SIZE </pre>	Main invariant(s) decimals is 18 and totalSupply always equals the amount minted at deployment Access level Public read-only What this function can do Read public state What this function cannot/should not do Change any state

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

TBCToken is not a vault and is not meant to hold value. It keeps the balance ledger for its own token, and the tokens themselves sit with the holders. The one place value can collect inside the contract is by accident.

Contract	Asset Type	Custodied Assets
TBCToken.sol	ERC20 (its own token)	None by design. Any tokens sent directly to the contract's own address stay there permanently, because there is no owner and no rescue function (Informational issue 1). The contract has no receive or fallback function, so plain Ether sent to it is rejected
Treasury address (off-chain multi-sig)	ERC20 (its own token)	The full supply at deployment, and whatever is left undistributed after the batchTransfer airdrop

3.2 Asset Entry & Exit Functions

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
TBCToken.sol	constructor	—	Full supply minted to treasury_	Deployer	The whole supply lands on one address and can never be minted again. A wrong treasury_ can only be fixed by redeploying before distribution starts
TBCToken.sol	transfer	Caller's balance	Recipient's balance	Public	Reverts for the zero address. The token's own address is accepted, and tokens sent there are lost (Informational issue 1)
TBCToken.sol	transferFrom	Approved holder's balance	Recipient's balance	Public, needs an allowance	Spending is capped by the allowance. An unlimited allowance is never reduced, so helper contracts such as Disperse should be approved only for the amount being sent

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
TBCToken.sol	batchTransfer	Caller's balance	Up to 300 recipients	Public	The whole call reverts if one leg fails. Amounts are in raw units (1 TBC = 10^{18}). Recipient lists should leave out contract addresses that cannot move ERC-20 tokens, including the token's own address (Informational issue 1)
TBCToken.sol	permit	—	Sets an allowance, moves no tokens	Public, needs the holder's signature	Each signature works once, and only on this token and this chain
TBCToken.sol	(no function)	Tokens sent to the contract address	—	Anyone	Nothing can move them out again. There is no rescue path

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of TBC Token. We performed our audit according to the procedure described above.

1 Informational Issue was found.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

October 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com